

1480. Running Sum of 1d Array

Easy Topics Companies Hint

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]...nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `[1,3,6,10]`
Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`
Output: `[1,2,3,4,5]`
Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`
Output: `[3,4,6,16,17]`

Constraints:

- 1 ≤ `nums.length` ≤ 1000
- 10⁶ ≤ `nums[i]` ≤ 10⁶



Code

Java Auto

```
1 class Solution {
2     public int[] runningSum(int[] nums) {
3         for (int i = 1; i < nums.length; i++) {
4             nums[i] += nums[i - 1];
5         }
6         return nums;
7     }
8 }
9
```

Saved

In 9, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`nums =`
`[1,2,3,4]`

Output

`[1,3,6,10]`

Expected

`[1,3,6,10]`

Contribute a testcase

27. Remove Element

Easy Topics Companies Hint

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` **in-place**. The order of the elements may be changed. Then return *the number of elements in `nums` which are not equal to `val`*.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
// It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [3,2,2,3], val = 3`
Output: `2, nums = [2,_,_,_]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being `2`. It does not matter what you leave beyond the returned `k` (hence they are underscores).

Code

```
Java Auto
1 class Solution {
2     public int removeElement(int[] nums, int val) {
3         int k = 0; // Pointer to place the next valid element
4
5         for (int i = 0; i < nums.length; i++) {
6             // If current element is not the target value
7             if (nums[i] != val) {
8                 nums[k] = nums[i]; // Move it to the front
9                 k++; // Increment the valid count
10            }
11        }
12
13        return k; // Return new effective length
14    }
15 }
16
```

Saved

Ln 16, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[3,2,2,3]

val =
3

Output

[2,2]

Expected

[2,2]

1470. Shuffle the Array

Easy | Topics | Companies | Hint

Given the array `nums` consisting of $2n$ elements in the form $[x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n]$.

Return the array in the form $[x_1, y_1, x_2, y_2, \dots, x_n, y_n]$.

Example 1:

Input: `nums = [2,5,1,3,4,7]`, `n = 3`
Output: `[2,3,5,4,1,7]`
Explanation: Since $x_1=2$, $x_2=5$, $x_3=1$, $y_1=3$, $y_2=4$, $y_3=7$ then the answer is `[2,3,5,4,1,7]`.

Example 2:

Input: `nums = [1,2,3,4,4,3,2,1]`, `n = 4`
Output: `[1,4,2,3,3,2,4,1]`

Example 3:

Input: `nums = [1,1,2,2]`, `n = 2`
Output: `[1,2,1,2]`

Constraints:

- $1 \leq n \leq 500$
- `nums.length == 2n`
- $1 \leq \text{nums}[i] \leq 10^3$

Discover more | Programming | Computer Science

Code

```
Java | Auto
class Solution {
    public int[] shuffle(int[] nums, int n) {
        int[] result = new int[2 * n];
        for (int i = 0; i < n; i++) {
            result[2 * i] = nums[i];
            result[2 * i + 1] = nums[i + n];
        }
        return result;
    }
}
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`nums =`
`[2,5,1,3,4,7]`

`n =`
`3`

Output

`[2,3,5,4,1,7]`

Expected

`[2,3,5,4,1,7]`

Change Theme Language Java 7

```
1 import java.io.*;
2 import java.util.*;
3 import java.text.*;
4 import java.math.*;
5 import java.util.regex.*;
6
7 public class Solution {
8
9     public static void main(String[] args) {
10         Scanner s = new Scanner(System.in);
11         int t = s.nextInt();
12         String [] pair_left = new String[t];
13         String [] pair_right = new String[t];
14
15         for (int i = 0; i < t; i++) {
16             pair_left[i] = s.next();
17             pair_right[i] = s.next();
18         }
19
20         //Write your code here
21
22         HashSet<String> uniquePairs = new HashSet<>(t);
23
24         for (int i = 0; i < t; i++) {
25             // Concatenate the left and right strings with a space delimiter
26             uniquePairs.add(pair_left[i] + " " + pair_right[i]);
27
28             // Print the current total of unique items inside the set
29             System.out.println(uniquePairs.size());
30         }
31         // -----
32
33         s.close();
34     }
35 }
36
37 >
```

```

1 import java.io.*;
2 import java.util.*;
3
4 public class Solution {
5     public static void main(String[] args) throws Exception {
6         BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
7
8         StringTokenizer st = new StringTokenizer(br.readLine());
9         int n = Integer.parseInt(st.nextToken());
10        int k = Integer.parseInt(st.nextToken());
11
12        int[] arr = new int[n];
13
14        st = new StringTokenizer(br.readLine());
15        for (int i = 0; i < n; i++) {
16            arr[i] = Integer.parseInt(st.nextToken());
17        }
18
19        HashMap<Integer, Integer> map = new HashMap<Integer, Integer>();
20
21        // Add the first window
22        for (int i = 0; i < k; i++) {
23            int count = map.containsKey(arr[i]) ? map.get(arr[i]) : 0;
24            map.put(arr[i], count + 1);
25        }
26
27        int maxUnique = map.size();
28
29        // Slide the window
30        for (int i = k; i < n; i++) {
31
32            // Remove the element leaving the window
33            int left = arr[i - k];
34            int count = map.get(left);
35
36            if (count == 1) {
37                map.remove(left);
38            } else {

```

Line: 58 Col: 1

 Upload Codes as File

☐ Test against custom input

Run Code

Submit Code

1732. Find the Highest Altitude

Easy | Topics | Companies | Hint

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length n where `gain[i]` is the **net gain in altitude** between points i and $i + 1$ for all $(0 \leq i < n)$. Return the **highest altitude** of a point.

Example 1:

Input: `gain = [-5,1,5,0,-7]`

Output: 1

Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:

Input: `gain = [-4,-3,-2,-1,4,3,2]`

Output: 0

Explanation: The altitudes are `[0,-4,-7,-9,-10,-6,-3,-1]`. The highest is 0.

Constraints:

- $n == \text{gain.length}$
- $1 \leq n \leq 100$
- $-100 \leq \text{gain}[i] \leq 100$



Seen this question in a real interview before? 1/6

3.5K | 428 | | |

Submit

Code

```
Java | Auto
class Solution {
    public int largestAltitude(int[] gain) {
        int currentAltitude = 0;
        int highestAltitude = 0;

        for (int g : gain) {
            currentAltitude += g;
            highestAltitude = Math.max(highestAltitude, currentAltitude);
        }

        return highestAltitude;
    }
}
```

Saved

Ln 14, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

gain =
[-5,1,5,0,-7]

Output

1

Expected

1

Contribute a testcase

Activate Windows
Go to Settings to activate Windows.

 0

Code

Java Auto

```

1 import java.util.*;
2
3 public class Solution {
4     public int[] topKFrequent(int[] nums, int k) {
5         // Step 1: Count the frequency of each element
6         Map<Integer, Integer> frequencyMap = new HashMap<>();
7         for (int num : nums) {
8             frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
9         }
10
11         // Step 2: Create buckets where index represents the frequency
12         // The maximum possible frequency is nums.length
13         List<Integer>[] buckets = new List[nums.length + 1];
14         for (int key : frequencyMap.keySet()) {
15             int frequency = frequencyMap.get(key);
16             if (buckets[frequency] == null) {
17                 buckets[frequency] = new ArrayList<>();
18             }
19             buckets[frequency].add(key);
20         }
21
22         // Step 3: Collect the top k elements
23         List<Integer> result = new ArrayList<>();
24         for (int i = buckets.length - 1; i >= 0; i--) {
25             List<Integer> bucket = buckets[i];
26             if (bucket == null) continue;
27             for (int num : bucket) {
28                 result.add(num);
29                 if (result.size() == k) return result.toArray(new Integer[k]);
30             }
31         }
32         return result.toArray(new Integer[k]);
33     }
34 }

```

Saved

Ln 40, Col: 1

Testcase > Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

```
nums =
[1,1,1,2,2,3]
```

 $k =$

2

Output

- [1, 2]

Expected

● 456 Online

 32°C
Light rain

49. Group Anagrams

Medium 🔖 Topics 🏷 Companies

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input: `strs = [""]`

Output: `[[""]]`

Example 3:

Input: `strs = ["a"]`

Output: `[["a"]]`

Constraints:

- $1 \leq \text{strs.length} \leq 10^4$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` consists of lowercase English letters.

👍 22.4K 🔄 426 ⭐ 15 📄 10

</> Code

Java 📄 Auto

```
1 import java.util.*;
2
3 public class Solution {
4     public List<List<String>> groupAnagrams(String[] strs) {
5         // Handle edge case
6         if (strs == null || strs.length == 0) {
7             return new ArrayList<>();
8         }
9
10        // Map to store sorted string signature as key and list of anagrams as value
11        Map<String, List<String>> map = new HashMap<>();
12
13        for (String s : strs) {
14            // Convert string to character array and sort it
15            char[] chars = s.toCharArray();
16            Arrays.sort(chars);
```

Saved

Ln 29, Col 1

📄 Testcase ➡️ Test Result

Accepted Runtime: 1 ms

✅ Case 1 ✅ Case 2 ✅ Case 3

Input

`strs =`
`["eat","tea","tan","ate","nat","bat"]`

Output

`[["eat","tea","ate"],["bat"],["tan","nat"]]`

Expected

`[["bat"],["nat","tan"],["ate","eat","tea"]]`

Activate Windows
Go to Settings to activate Windows.

👍 Contribute a testcase

👤 560 Online

26. Remove Duplicates from Sorted Array

Easy Topics Companies Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [1,1,2]`
Output: `2`, `nums = [1,2,_]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 1 and 2 respectively. It does not matter what you leave beyond the returned `k` (hence they are underscores).

Example 2:

Input: `nums = [0,0,1,1,1,2,2,3,3,4]`
Output: `5`, `nums = [0,1,2,3,4,_,_,_,_,_,_]`

19.6K 1.1K 1 1

Submit

Code

```
Java Auto
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         if (nums.length == 0) {
4             return 0;
5         }
6
7         // Pointer to track the index of the next unique element
8         int insertIndex = 1;
9
10        // Iterate through the array starting from the second element
11        for (int i = 1; i < nums.length; i++) {
12            // Found a new unique element
13            if (nums[i] != nums[i - 1]) {
14                nums[insertIndex] = nums[i];
15                insertIndex++;
16            }
17        }
18    }
19 }
```

Saved

Ln 23, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[1,1,2]

Output

[1,2]

Expected

[1,2]

Activate Windows
Go to Settings to activate Windows.

Contribute a testcase

575 Online